
Redyx — Vote Service Test Cases

idityaGE

2026-05-10

Contents

Vote Service — Test Case Specification **2**

 Test Infrastructure 2

 1. VoteStore — Lua Script Vote Transition Tests 2

 2. VoteStore — GetVoteState Tests 3

 3. VoteStore — GetScore Tests 4

 4. VoteStore — BatchGetVoteStates Tests 4

 5. Server Input Validation Tests (Vote RPC) 4

 6. GetVoteState Server Tests 5

 7. toInt Helper Tests 6

 Vote Transition Matrix (Lua Script Coverage) 6

 Patterns & Conventions 6

Vote Service — Test Case Specification

Project Name	Redyx
Module Name	Vote
File	internal/vote/server_test.go
Total Tests	30
Package	vote (same-package test)
Created By	idityaGE
Date of Creation	2026-05-10

Test Infrastructure

Component	Implementation
Logger	<code>zap.NewNop()</code>
Redis	<code>miniredis.RunT(t)</code> (in-process)
Kafka Producer	<code>nil</code> — vote logic tested via <code>VoteStore</code> ; publish panics are recovered
Auth Context	<code>auth.WithClaims(context.Background(), &auth.Claims{...})</code>
Helper convention	<code>t.Helper()</code> in <code>testRedis</code> , <code>testVoteServer</code> , <code>authedVoteCtx</code>
Panic recovery	<code>defer recover()</code> in server tests where <code>nil</code> Kafka producer may panic at publish step

1. VoteStore — Lua Script Vote Transition Tests

The `VoteStore` uses a Redis Lua script to atomically handle all 9 vote transitions (none/up/down × none/up/down).

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_001	<code>TestCastVote_IncreasesScore</code>	<code>miniredis</code> running; no prior vote	Call <code>store.CastVote(cfirst vote, "user-1", "post-1", "up")</code>	<code>direction = "up"</code>	<code>delta=1, newScore=1</code>	PASS

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_002	TestCastVote_Downvote_De-createsScore	miniredis running; no prior vote	Call <code>store.CastVote("user-1", "post-1", "down")</code>	direction = "down" (first vote)	delta=-1, newScore=-1	PASS
TC_VOTE_003	TestCastVote_Idempotent_SameDirection	miniredis running; first upvote cast	Call <code>store.CastVote</code> a second time with same "up" direction	direction = "up" twice	Second call: delta=0 (no-op; idempotent)	PASS
TC_VOTE_004	TestCastVote_FlipUpToDown_DeltaMinusTwo	miniredis running; prior upvote cast	Call <code>store.CastVote</code> with "down" after "up"	upvote → downvote flip	delta=-2 (removes +1 and adds -1)	PASS
TC_VOTE_005	TestCastVote_FlipDownToUp_DeltaPlusTwo	miniredis running; prior downvote cast	Call <code>store.CastVote</code> with "up" after "down"	downvote → upvote flip	delta=2 (removes -1 and adds +1)	PASS
TC_VOTE_006	TestCastVote_RemoveUpvote_DeltaMinusOne	miniredis running; prior upvote cast	Call <code>store.CastVote</code> with "none" after "up"	upvote → none	delta=-1, newScore=0	PASS
TC_VOTE_007	TestCastVote_NoneToNone_Idempotent	miniredis running; no prior vote	Call <code>store.CastVote</code> with "none" (no existing vote)	none → none	delta=0 (no-op)	PASS
TC_VOTE_008	TestCastVote_MultipleUsers_IndependentScores	miniredis running	Three users each upvote "post-1" independently	user-1, user-2, user-3 all upvote post-1	After all three votes: newScore=3	PASS

2. VoteStore — GetVoteState Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_009	TestGetVoteState_ReturnsCorrectState	miniredis running	1. Check state before any vote 2. Upvote; check state 3. Remove vote; check state	user-1, post-1	1. State = "" (no vote); 2. State = "up"; 3. State = "" (vote removed)	PASS

3. VoteStore — GetScore Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_010	TestGetScore_NoVotes_ReturnsZero	miniredis running; no votes cast	Call <code>store.GetScore(ctx, votes)</code>	<code>target = "post-1", votes</code>	Returns <code>score=0</code> , no error	PASS

4. VoteStore — BatchGetVoteStates Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_011	TestBatchGetVoteStates_Empty	miniredis running	Call <code>store.BatchGetVoteStates(ctx, "user-1", [])</code> with empty targetIDs	<code>targetIds = []</code>	Returns empty map, no error	PASS
TC_VOTE_012	TestBatchGetVoteStates_MultipleTargets	miniredis running; user-1 upvoted post-1, downvoted post-2; post-3 unvoted	Call <code>store.BatchGetVoteStates(ctx, "user-1", ["post-1", "post-2", "post-3"])</code>	user-1 with 3 targets	<code>states["post-1"]="up"</code> , <code>states["post-2"]="down"</code> , <code>states["post-3"]=""</code>	PASS

5. Server Input Validation Tests (Vote RPC)

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_013	TestVote_Unauthenticated	Vote server initialized	Call Vote with unauthenticated context	<code>targetId = "post-1"</code> , type = POST, direction = UP	Returns an error (missing auth claims)	PASS
TC_VOTE_014	TestVote_EmptyTargetID	Authenticated context	Call Vote with empty target_id	<code>targetId = ""</code> , type = POST, direction = UP	Returns an error	PASS
TC_VOTE_015	TestVote_InvalidTargetType	Authenticated context	Call Vote with <code>TARGET_TYPE_UNSPECIFIED</code>	<code>targetId = "post-1"</code> , type = UNSPECIFIED, direction = UP	Returns an error	PASS

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_016	TestVote_InvalidDirection	Authenticated context	Call Vote with VOTE_DIRECTION_UNSPECIFIED	targetId = "post-1", type = POST, direction = UNSPECIFIED	Returns an error	PASS
TC_VOTE_017	TestVote_ValidUpvote_ReturnsNewScore	Authenticated context; nil Kafka producer	Call Vote with valid upvote; recover Kafka panic	direction = UP, target = "post-1"	If no panic: resp.NewScore=1; if Kafka panics (recovered): vote logic validated via VoteStore tests	PASS
TC_VOTE_018	TestVote_ValidDownvote_ReturnsNegativeScore	Authenticated context; nil Kafka producer	Call Vote with valid downvote; recover Kafka panic	direction = DOWN, target = "post-2"	If no panic: resp.NewScore=1	PASS
TC_VOTE_019	TestVote_None_RemovesVote	Authenticated context; nil Kafka producer	1. Upvote post-3 (recover panic); 2. Cast NONE to remove vote	direction = NONE, target = "post-3"	If no panic: resp.NewScore=0	PASS

6. GetVoteState Server Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_020	TestGetVoteState_Unauthenticated	Vote server initialized	Call GetVoteState with unauthenticated context	targetId = "post-1"	Returns an error	PASS
TC_VOTE_021	TestGetVoteState_NoVote_ReturnsUnspecified	Authenticated context; no prior vote	Call GetVoteState for unvoted target	targetId = "post-never-voted"	resp.Direction = VOTE_DIRECTION_UNSPECIFIED	PASS
TC_VOTE_022	TestGetVoteState_AfterUpvote_ReturnsUp	Authenticated context; prior upvote on target	Call Vote(UP); call GetVoteState	targetId = "post-state-1"	resp.Direction = VOTE_DIRECTION_UP	PASS
TC_VOTE_023	TestGetVoteState_AfterDownvote_ReturnsDown	Authenticated context; prior downvote on target	Call Vote(DOWN); call GetVoteState	targetId = "post-state-2"	resp.Direction = VOTE_DIRECTION_DOWN	PASS

7. toInt Helper Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_VOTE_024	TestToInt_Int64	—	Call <code>toInt(int64(42))</code>	value = <code>int64(42)</code>	Returns 42, no error	PASS
TC_VOTE_025	TestToInt_String	—	Call <code>toInt("99")</code>	value = "99"	Returns 99, no error	PASS
TC_VOTE_026	TestToInt_InvalidString	—	Call <code>toInt("not-a-number")</code>	value = "not-a-number"	Returns an error (parse failure)	PASS
TC_VOTE_027	TestToInt_InvalidType	—	Call <code>toInt(struct{})</code>	value = <code>struct{}</code>	Returns an error (unsupported type)	PASS

Vote Transition Matrix (Lua Script Coverage)

Previous Vote	New Vote	Expected Delta	Test Case
none	up	+1	TC_VOTE_001
none	down	-1	TC_VOTE_002
none	none	0	TC_VOTE_007
up	up	0 (idempotent)	TC_VOTE_003
up	down	-2	TC_VOTE_004
up	none	-1	TC_VOTE_006
down	up	+2	TC_VOTE_005
down	down	0 (idempotent)	— (covered by TC_VOTE_003 logic)
down	none	+1	— (covered by TC_VOTE_006 logic)

Patterns & Conventions

Pattern	Implementation
Package scope	<code>package.vote</code> (white-box / same-package)
Logger	<code>zap.NewNop()</code> (silent)
In-process Redis	<code>miniredis.RunT(t)</code> — auto-cleaned on <code>t.Cleanup</code>
Nil Kafka tolerance	Panic from nil Kafka producer is recovered in server-level tests

Pattern	Implementation
Direction mapping	"up" → VOTE_DIRECTION_UP, "down" → VOTE_DIRECTION_DOWN, "" → VOTE_DIRECTION_UNSPECIFIED
Auth context	auth.WithClaims(context.Background(), &auth.Claims{UserID, Username})
Helper tagging	t.Helper() in testRedis, testVoteServer, authedVoteCtx